

MATEMÁTICA COMPUTACIONAL APLICADA

Aula 09

Introdução a Circuitos Digitais

Da álgebra booleana às portas físicas — e do meio somador ao coração de qualquer processador.

AND

OR

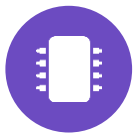
NOT

NAND

NOR

XOR

Nesta aula



Portas lógicas e seus símbolos

Cada operação booleana, agora como componente físico.



Da tabela-verdade ao circuito

O caminho completo: especificação → expressão → circuito.



Meio somador (half adder)

O primeiro circuito aritmético — soma dois bits.



Por que álgebra booleana é a base do hardware

Cada literal simplificado vira, literalmente, uma porta a menos.

ANTES DE COMEÇAR

O que já sabemos

Axiomas booleanos, simplificação algébrica e mapas de Karnaugh já são ferramentas conhecidas.

$$x + y$$

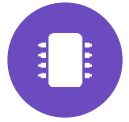
$$x \cdot y$$

$$x^-$$

três operações, sempre sobre $\{0, 1\}$



Hoje: cada uma dessas operações ganha um corpo físico — a porta lógica.



Portas lógicas

Cada operação booleana da Aula 06 vira uma porta física:

Porta	Símbolo	Função
AND	$\wedge$	1 só se todas entradas = 1
OR	$\vee$	1 se alguma entrada = 1
NOT	$\neg$	inverte
NAND	$\overline{\wedge}$	NOT(AND)
NOR	$\overline{\vee}$	NOT(OR)
XOR	$\oplus$	1 se entradas diferentes

UM FATO SURPREENDENTE



NAND é universal

Qualquer circuito pode ser construído só com portas NAND (ou só NOR). Isso simplifica a fabricação de chips: uma linha de produção especializada numa única porta.

PROVANDO O FATO

Por que NAND é universal

Escrevendo $\text{NAND}(x,y) = (x \cdot y)^{-}$, conseguimos montar NOT, AND e OR só com ela:

NOT

$$x^{-} = \text{NAND}(x, x)$$

$$(x \cdot x)^{-} = \bar{x}$$

AND

$$xy = \text{NAND}(\text{NAND}(x, y), \text{NAND}(x, y))$$

NAND seguido de NOT

OR

$$x + y = \text{NAND}(x^{-}, \bar{y})$$

De Morgan: $(\bar{x} \cdot \bar{y})^{-} = x + y$

CONFIRMANDO

AND e OR, construídos só com NAND

As colunas construídas batem exatamente com AND e OR de verdade.

x	y	NAND	AND via NAND	OR via NAND
1	1	0	1	1
1	0	1	0	1
0	1	1	0	1
0	0	1	0	0

Quatro linhas, duas fórmulas confirmadas — sem precisar de nenhuma outra porta física.

Da lógica ao circuito

Um circuito combinacional implementa uma função booleana — o fluxo conecta tudo o que vimos até aqui.



Cada seta já foi construída em uma aula diferente — hoje fechamos o ciclo.

Meio somador (Half Adder)

Soma dois bits A e B, produzindo a soma S e o "vai-um" (carry) C:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A \oplus B$$

$$C = A \cdot B$$

0 meio somador em portas

As mesmas entradas A, B alimentam duas portas diferentes, em paralelo:



Duas portas, duas saídas — nenhuma delas depende da outra.



Nada disso é novo

S é exatamente o XOR que já vimos na Aula 06, e C é um simples AND. O meio somador nada mais é do que duas portas já conhecidas, combinadas.

`meio somador = XOR + AND, sem nada mais`



Simulando o meio somador

Antes de existir em silício, todo circuito é simulado em código — as mesmas operações bit a bit que já usamos desde a Aula 06.

half_adder / HalfAdder

XOR e AND, exatamente como nas expressões do Slide 9.

HalfAdder.cs



```
static (int soma, int carry) HalfAdder(  
    int a, int b)  
{  
    int soma = a ^ b;  
    int carry = a & b;  
    return (soma, carry);  
}
```

half_adder.py



```
def half_adder(a, b):  
    soma = a ^ b      # XOR  
    carry = a & b     # AND  
    return soma, carry  
  
for a in (0, 1):  
    for b in (0, 1):  
        print(a, b, "->", half_adder(a, b))
```



Somador completo (Full Adder)

O meio somador não considera um "vai-um" de entrada. O full adder soma A, B e Cin:

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Expressões do full adder

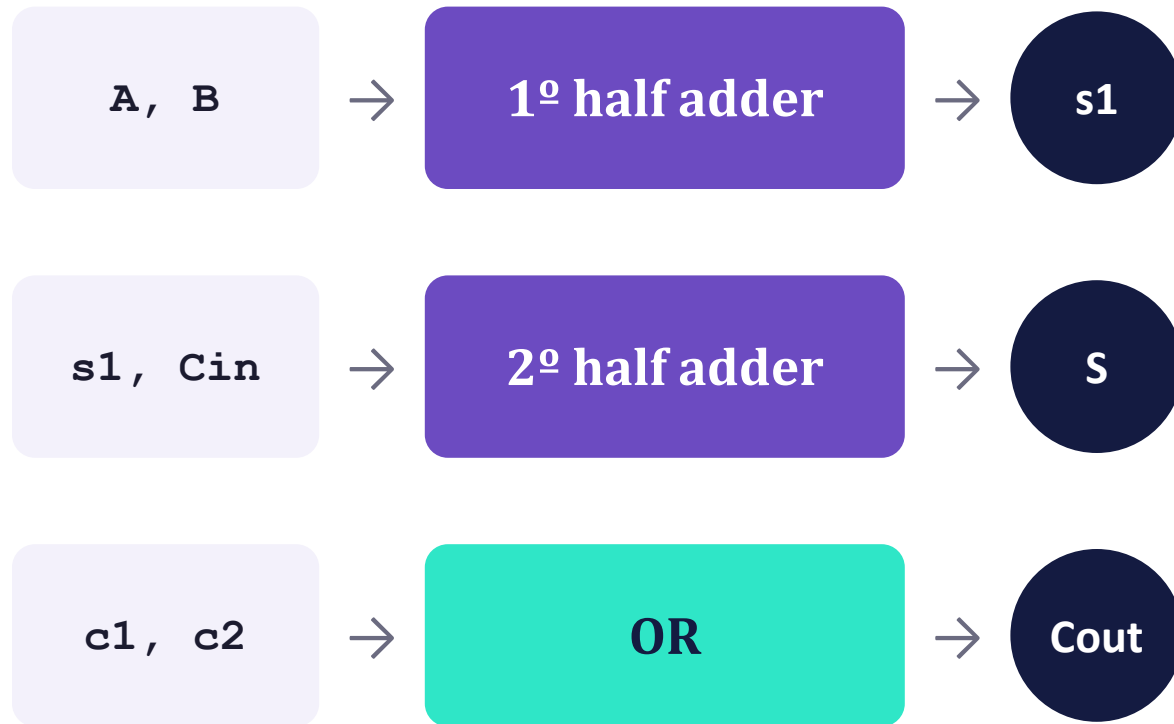
$$S = A \oplus B \oplus Cin$$

$$Cout = AB + Cin(A \oplus B)$$

Cout vale 1 quando A e B já são 1, ou quando o carry de entrada "empurra" a soma $A \oplus B$ para 1.

Full adder = dois half adders

Encadeando dois meios somadores e um OR para combinar os carries:



c1 = carry do 1º half adder c2 = carry do 2º half adder



Da soma de 1 bit à ULA

Encadeando n full adders — o Cout de um vira o Cin do próximo — construímos somadores de n bits: a base da ULA (unidade lógica e aritmética) de qualquer processador.



Cada bit soma o que sobrou do bit anterior — exatamente como somar à mão, coluna por coluna.



Aplicações

ULA / CPU

Somadores, comparadores e multiplexadores são circuitos booleanos.

Memórias e registradores

Portas realimentadas guardam bits.

Controle

Decodificadores traduzem instruções em sinais.



Hora de praticar

Quatro exercícios: circuito, meio somador, NAND universal e um desafio com full adder.



EXERCÍCIO 1

Desenhando o circuito

Desenhe (ou descreva) o circuito de $f = \bar{A} \cdot B + A \cdot \bar{B}$. Que porta única equivale a ele?

$$f = \bar{A} \cdot B + A \cdot \bar{B}$$

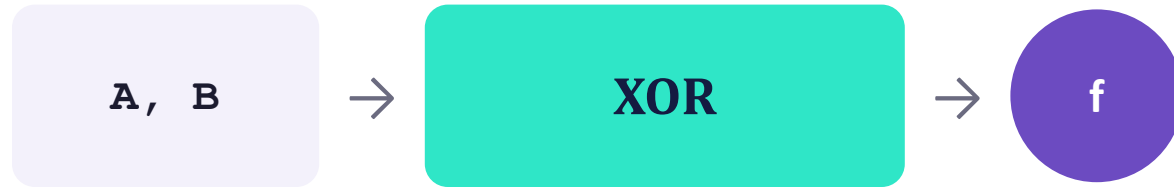
Circuito (portas usadas):

Porta única equivalente: _____



GABARITO · EXERCÍCIO 1

Desenhando o circuito



$\bar{A} \cdot B + A \cdot \bar{B}$ é exatamente a definição de XOR — o circuito inteiro colapsa numa única porta.

Porta equivalente: XOR



EXERCÍCIO 2

Tabela do meio somador

Monte a tabela-verdade do meio somador e confira contra a função Python da aula.

A	B	S	C
0	0	?	?
0	1	?	?
1	0	?	?
1	1	?	?



GABARITO · EXERCÍCIO 2

Tabela do meio somador

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Idêntica ao Slide 9 e à saída de `half_adder(a, b)` para as 4 combinações.



EXERCÍCIO 3

NAND universal

Mostre que NAND é universal: implemente NOT, AND e OR usando apenas NAND, em Python (defina só `def nand(x, y): return 1 - (x & y)` e construa o resto a partir dela).

```
● ● ● nand_universal.py

def nand(x, y): return 1 - (x & y)

def not_(x): return _____

def and_(x, y): return _____

def or_(x, y): return _____
```




GABARITO · EXERCÍCIO 3

NAND universal

 nand_universal.py



```
def nand(x, y): return 1 - (x & y)

def not_(x): return nand(x, x)

def and_(x, y): return nand(nand(x, y), nand(x, y))

def or_(x, y): return nand(not_(x), not_(y))
```

Equivalente em C#: Nand, Not, And e Or seguem a mesma composição, linha a linha.



EXERCÍCIO 4 · DESAFIO

Full adder de verdade

Implemente `full_adder(a, b, carry_in)` encadeando duas chamadas de `half_adder`. Teste todas as 8 combinações de `a`, `b`, `carry_in`.

dica: veja o diagrama do Slide 16 — dois `half_adder` e um OR nos carries



`full_adder.py`

```
def full_adder(a, b, carry_in):
```

```
    s1, c1 = half_adder(a, b)
```

```
    s2, c2 = _____
```

```
    return _____
```



GABARITO · EXERCÍCIO 4

Full adder de verdade

full_adder.py



```
def full_adder(a, b, carry_in):  
    s1, c1 = half_adder(a, b)  
    s2, c2 = half_adder(s1, carry_in)  
    cout = c1 | c2  
    return s2, cout
```

Teste: for a in (0,1): for b in (0,1): for cin in (0,1): print(a,b,cin,full_adder(a,b,cin))

c1 e c2 nunca valem 1 ao mesmo tempo — por isso o | (OR) nunca perde informação.

Referências

Livros — teoria



TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L.

Sistemas Digitais: princípios e aplicações.
Pearson — portas lógicas e circuitos
combinacionais.



IDOETA, I. V.; CAPUANO, F. G.

Elementos de Eletrônica Digital. Érica — cap.
Circuitos combinacionais (meio somador e
somador completo).



NISAN, N.; SCHOCKEN, S.

The Elements of Computing Systems (Nand2Tetris)
— construindo um computador a partir de portas
NAND.
<https://www.nand2tetris.org/>

Documentação e vídeos



Python — operadores bit a bit

[^, & e | \(bitwise operations\)](#) — a base de todo circuito simulado.

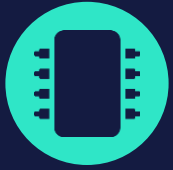


Vídeos

[Ben Eater — computador de 8 bits](#) — portas lógicas em hardware real.

[Nand2Tetris — half e full adder](#) — construindo a partir de NAND.

[Computerphile — por que NAND é universal](#)



FECHANDO O CICLO

Da proposição lógica ao circuito de verdade

Toda a matemática discreta até aqui já vira hardware.

Do XOR de duas proposições ao somador de um processador inteiro — é a mesma lógica, em escalas diferentes.